

Teknologi dan Arsitektur - Multi-Provider Object Storage SaaS

Keputusan Arsitektur

Gunakan:

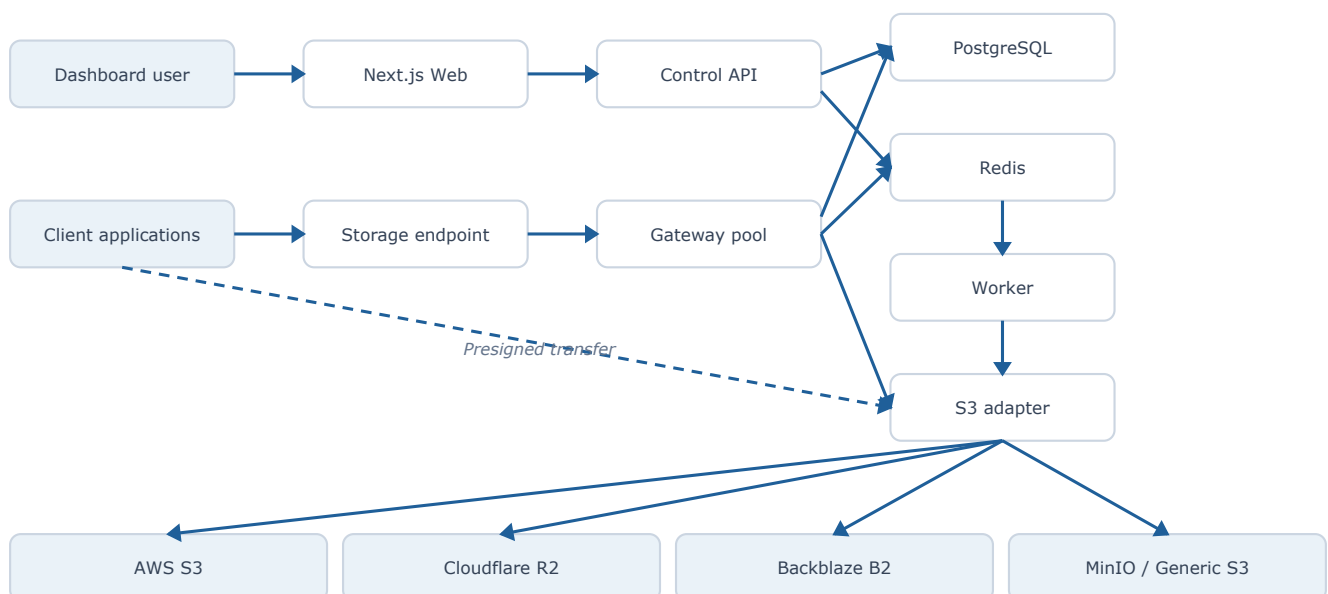
- Satu monorepo.
- Modular monolith untuk domain dan business logic.
- Runtime web dan API terpisah.
- Storage gateway yang dapat direplikasi secara horizontal.
- Worker terpisah untuk background jobs.
- PostgreSQL sebagai metadata source of truth.
- Redis untuk queue, distributed rate limit, dan koordinasi.

Jangan mulai dengan:

- Next.js full-stack monolith untuk seluruh trafik file.
- Microservices per domain.
- Seluruh stack terpisah untuk setiap tenant.
- Port berbeda sebagai mekanisme tenancy.

Arsitektur kode tetap sederhana, tetapi data plane dapat di-scale secara independen dari dashboard.

Arsitektur Tingkat Tinggi



Control plane

Control plane menangani:

- Authentication dan session.

- Organization dan membership.
- RBAC.
- Provider connection.
- Bucket connection.
- Virtual namespace.
- API credential.
- Folder grant.
- Placement policy.
- Cross-provider migration orchestration.
- Quota dan usage.
- Audit log.
- Platform administration.

Data plane

Data plane menangani:

- Authentication API client.
- List, HEAD, upload, download, dan delete object.
- Multipart upload.
- Prefix policy enforcement.
- Quota reservation.
- Provider placement.
- Cross-provider object streaming, verification, dan location cutover.
- Streaming atau presigned transfer.
- Checksum dan completion verification.
- Request metering dan rate limiting.

Struktur Repository

apps/	
web/	Next.js dashboard
api/	Express control API dan gateway entry point
worker/	Background jobs
packages/	
shared/	Zod schemas, DTO, permission keys
db/	Prisma schema dan Prisma client
storage-core/	Namespace, placement, quota, object lifecycle, migration
provider-s3/	S3 adapter dan provider capability probing
auth/	JWT, API credential, authorization helpers
observability/	Logger, metrics, tracing
config/	Typed runtime configuration

Runtime boundary:

- Web tidak mengakses database atau provider secara langsung.
- API dan worker boleh memakai packages/db.
- Provider SDK hanya dipakai melalui storage service boundary.

- UI tidak berisi placement, quota, atau authorization business rules.

Stack Teknologi

Area	Teknologi	Catatan
Package manager	npm workspaces	Cukup untuk monorepo awal; build orchestrator dapat ditambahkan bila diperlukan
Frontend	Next.js App Router dan TypeScript	Dashboard SaaS dan admin
UI	shadcn/ui	Komponen dashboard yang dapat dikustomisasi
API	Express dan TypeScript	Runtime terpisah dari Next.js
Validation	Zod	Shared request/response schemas
API specification	OpenAPI	Contract untuk dashboard, SDK, dan external clients
ORM	Prisma	Metadata dan transactional control
Database	PostgreSQL	Tenant data, logical object index, physical locations, usage, migration, audit
S3 SDK	AWS SDK for JavaScript v3	Provider-specific endpoint dan region
Queue	BullMQ	Reconciliation, cleanup, migration, provider health
Coordination	Redis	Rate limit, queue, short-lived locks, cache
Authentication	JWT access token dan rotated refresh token	Refresh token disimpan sebagai hash
Client credentials	Generated API key	Secret ditampilkan sekali; database menyimpan hash
Logging	Pino	Structured logging dengan redaction
Tracing	OpenTelemetry	Request dan provider-call tracing
Metrics	Prometheus-compatible metrics	Traffic, latency, errors, queue depth
Containers	Docker	Image terpisah per runtime
Reverse proxy	HAProxy, Nginx, atau Caddy	TLS, routing, request limits
Network shaping	OPNsense	Aggregate atau dedicated network-level cap
Tests	Vitest, Supertest, Playwright	Unit, API integration, dan critical UI flows
S3 test target	MinIO container dan live sandbox providers	Local contract test serta provider compatibility

Versi dependency harus dipin dan diperbarui secara terkontrol ketika scaffold dibuat. Dokumen ini sengaja tidak mengunci nomor versi sebelum implementasi dimulai.

Provider Adapter

Gunakan satu interface internal agar business logic tidak bergantung pada provider tertentu.

Contoh tanggung jawab adapter:

```
testConnection
listBuckets
headBucket
listObjects
headObject
putObjectStream
getObjectStream
getObjectRange
getObjectTags
putObjectTags
deleteObject
createMultipartUpload
uploadPart
completeMultipartUpload
abortMultipartUpload
createPresignedRequest
```

Setiap `ProviderConnection` menyimpan:

- Provider type.
- Endpoint.
- Region.
- Addressing mode: virtual-hosted atau path-style.
- Access key reference.
- Secret key encrypted payload atau secret reference.
- Optional session token.
- TLS policy.
- Capability flags.
- Last health status.

Capability probing perlu menguji:

- Signature V4.
- Path-style versus virtual-hosted-style request.
- Multipart upload.
- Presigned GET, PUT, HEAD, dan DELETE.
- Range request.
- Checksum behavior.
- Object versioning.
- Tagging.
- Server-side encryption options.
- Bucket CORS.

Tidak semua S3-compatible provider mempunyai behavior yang sama. Application layer harus menggunakan lowest common denominator atau memeriksa capability sebelum mengaktifkan fitur.

Mode Transfer

Managed direct

```
Client -> Control request -> Gateway
Gateway -> Placement and quota reservation
Gateway -> Presigned URL
Client -> S3 provider
Client -> Complete request -> Gateway
```

Kelebihan:

- Trafik file tidak melewati gateway.
- Lebih murah dan scalable.
- Provider credential tidak diberikan kepada client.

Kekurangan:

- Provider hostname dapat terlihat.
- OPNsense milik SaaS tidak dapat membentuk trafik langsung tersebut.

- Completion dan orphan cleanup harus ditangani.

Full proxy

```
Client -> OPNsense -> Gateway -> S3 provider
```

Kelebihan:

- Stable endpoint.
- Provider sepenuhnya disembunyikan.
- Application dan OPNsense dapat mengontrol throughput.
- Cocok untuk S3-compatible facade.

Kekurangan:

- Semua byte melewati infrastruktur SaaS.
- Membutuhkan kapasitas TLS, socket, bandwidth, dan high availability.
- Risiko timeout dan resource exhaustion lebih tinggi.

Hybrid

Rekomendasi:

- API dan SDK selalu memakai endpoint produk.
- Upload/download besar default ke presigned transfer.
- Full proxy tersedia untuk requirement tertentu.
- Enterprise dapat memakai dedicated proxy gateway.
- Semua metadata, authorization, quota, dan placement tetap melalui control plane.

Cross-Provider Migration Architecture

Migrasi current active object adalah fitur wajib MVP. Cross-provider CopyObject tidak diasumsikan tersedia karena provider sumber dan tujuan dapat berbeda. Data path default:

```
Source S3 provider
-> bounded stream atau range read
-> migration worker
-> multipart upload
-> destination S3 provider
```

Worker tidak boleh menyimpan full object di memory atau local disk. Buffer, multipart part size, concurrent objects, dan concurrent parts harus dibatasi melalui configuration serta plan entitlement.

Preflight dan scheduling

Control API harus:

1. Memvalidasi organization ownership, RBAC, source, destination, namespace, dan prefix scope.
2. Menjalankan health, credential, capability, capacity, quota, encryption, metadata, dan checksum preflight.
3. Mengarahkan write baru pada migration scope ke destination placement target.
4. Membuat MigrationRun dan MigrationItem idempotent.
5. Mengantri item menggunakan stable deduplication key berbasis migrationRunId dan objectRecordId.

Migration queue terpisah dari request-critical cleanup dan provider-health queue agar bulk copy tidak menghambat operasi interaktif. Scheduler harus mendukung pause, resume, retry-failed, cancel sebelum cutover, per-organization concurrency, dan per-provider concurrency.

Copy dan verification

Untuk setiap item, worker:

1. Membaca active ObjectLocation dan source fingerprint yang minimal mencakup location ID, size, checksum, serta provider version ID atau ETag dan Last-Modified bila tersedia.
2. Membuka streaming GET atau range read dari source.
3. Membuat candidate ObjectLocation dan multipart upload pada destination.
4. Menghitung atau meneruskan checksum independen seperti SHA-256 selama streaming. Prefer trusted checksum yang disimpan saat ingest.
5. Menyalin Content-Type dan custom metadata yang portable serta memetakan tags, storage class, dan encryption berdasarkan target capability.
6. Menyelesaikan multipart upload lalu menjalankan HEAD pada target.
7. Memverifikasi size dan checksum melalui target checksum API. Jika tidak ada trusted source checksum atau target checksum API, lakukan target read-back dan hitung ulang checksum sebelum cutover. ETag tidak digunakan sebagai checksum universal.

Candidate ObjectLocation tidak boleh digunakan untuk read sebelum verification berhasil.

Atomic cutover

Cutover dijalankan dalam transaction PostgreSQL:

1. Lock ObjectRecord.
2. Pastikan activeObjectLocationId dan source fingerprint belum berubah.
3. Ubah source ObjectLocation menjadi RETAINED.
4. Ubah candidate ObjectLocation menjadi ACTIVE.
5. Update ObjectRecord.activeObjectLocationId.
6. Tulis transactional outbox dan audit event.
7. Commit.

Jika source berubah akibat overwrite, item menjadi RETRY_PENDING dan disalin ulang dari active version terbaru. Jika object dihapus, tombstone atau terminal state mencegah worker menghidupkan kembali object.

Read selalu mengikuti activeObjectLocationId. Kegagalan sebelum commit tidak mengubah read path. Setelah commit, retained source memungkinkan rollback yang diaudit selama retention window.

Cleanup dan accounting

- Logical usedBytes tidak bertambah karena temporary double storage.
- Physical migration bytes, source-retained bytes, request count, dan provider egress dicatat terpisah untuk capacity dan cost visibility.
- Source tidak dihapus sebelum retention window, presigned URL expiry, dan reconciliation selesai.
- Cleanup menggunakan outbox, idempotent delete, dan retry dengan bounded exponential backoff.
- Cancel sebelum cutover membuat item terminal dan membersihkan candidate object atau multipart upload secara idempotent.

- MigrationRun selesai hanya setelah semua item terminal dan cleanup policy terpenuhi.
- MVP memigrasikan current active object saja. Historical version migration, permanent replication, dan automatic cost-based migration adalah workstream terpisah.

Bandwidth Architecture

Lapisan kontrol

1. Product quota
 - Stored bytes.
 - Reserved bytes.
 - Monthly egress.
 - Request count.
 - Maximum object size.
 - Migration bytes dan retained-source bytes sebagai physical usage terpisah.
2. Application rate limit
 - organizationId.
 - apiCredentialId.
 - namespaceId.
 - Concurrent upload/download.
 - Requests per second.
 - Optional bytes per second.
 - Migration bytes per second dan migration concurrency.
3. Gateway dan worker backpressure
 - Maximum active streams.
 - Per-provider concurrency.
 - Per-organization migration concurrency.
 - Multipart part concurrency dan bounded buffer.
 - Queue depth.
 - Socket and memory limits.
4. OPNsense shaping
 - Aggregate gateway pool cap.
 - Business-tier pool cap.
 - Dedicated tenant private IP/VIP cap.

OPNsense tidak menjadi source of truth untuk SaaS entitlement. Source of truth tetap plan, quota, dan usage record di aplikasi.

Migration worker yang melewati OPNsense dapat diberi aggregate atau dedicated network cap. Precise per-organization migration rate tetap diterapkan di worker karena OPNsense tidak memahami organizationId dalam encrypted object traffic.

Shared gateway pool

Contoh:

```
storage.example.com
-> 10.20.10.11 gateway-1
-> 10.20.10.12 gateway-2
-> 10.20.10.13 gateway-3
```

Semua replica stateless dan melayani banyak tenant. Redis menyimpan distributed limits. OPNsense membatasi total pool, misalnya 1 Gbps.

OPNsense tidak dapat mengetahui tenant dari API key yang berada dalam HTTPS. Karena itu, precise per-tenant rate limiting tetap dilakukan di gateway.

Business tier pool

Contoh:

```
standard.storage.example.com -> standard pool
business.storage.example.com -> business pool
```

Setiap pool memiliki:

- Stable VIP.
- Replica count sendiri.
- OPNsense pipe sendiri.
- Application plan limits sendiri.

Model ini lebih efisien daripada deployment per tenant dan memberi pemisahan kapasitas antar-plan.

Dedicated enterprise gateway

Contoh:

```
tenant-a.storage.example.com -> 10.20.30.11
tenant-b.storage.example.com -> 10.20.30.12
```

Yang diduplikasi:

- Gateway deployment.
- Stable private IP atau VIP.
- Tenant-scoped runtime configuration.
- Optional dedicated worker concurrency.
- OPNsense pipe dan rules.

Yang tetap shared:

- Dashboard.
- Control API.
- PostgreSQL, kecuali ada requirement database isolation.
- Redis, kecuali ada requirement dedicated coordination.
- Global admin.

Gateway khusus tetap wajib memvalidasi organizationId dari credential. Network routing bukan authorization.

Mengapa tidak memakai port per tenant

- Sulit dioperasikan pada skala besar.

- S3 client umumnya mengharapkan HTTPS 443.
- Firewall, certificate, dan monitoring menjadi rumit.
- Port bukan security boundary.
- Port tidak menyelesaikan tenant isolation.

Gunakan standard port 443, hostname, credential, dan stable VIP.

Jangan membangun OPNsense rules berdasarkan ephemeral container IP. Gunakan:

- Load balancer VIP.
- Gateway node atau VM IP.
- Kubernetes service IP atau LoadBalancer IP.
- Dedicated VLAN interface.

Deployment Model

SaaS tier	Web/control plane	Gateway	Database	OPNsense
Free/Standard	Shared	Shared pool	Shared	Aggregate pool cap
Business	Shared	Dedicated tier pool	Shared	Cap per tier pool
Enterprise	Shared	Dedicated tenant gateway	Shared by default	Pipe per tenant VIP
Regulated	Dedicated optional	Dedicated	Dedicated optional	Dedicated network zone
On-premise	Customer deployment	Customer deployment	Customer deployment	Customer-controlled

Menduplikasi seluruh stack hanya masuk akal untuk regulated, data residency, atau on-premise offering.

Database dan Consistency

PostgreSQL adalah source of truth untuk:

- Tenant ownership.
- Logical key.
- Active, candidate, dan retained physical object locations.
- Object state.
- Migration run dan per-object migration state.
- Quota reservations.
- Usage counters.
- API credentials.
- Audit metadata.

PostgreSQL dan S3 tidak dapat masuk dalam satu ACID transaction. Gunakan:

- Idempotency key.
- Explicit state machine.
- Transactional outbox.
- Retry dengan bounded exponential backoff.
- Reconciliation jobs.

- Orphan upload cleanup.

ObjectRecord menyimpan logical identity, checksum, logical timestamps, dan activeObjectLocationId. ObjectLocation menyimpan provider, bucket, physical key, ETag, provider version, physical Last-Modified, verification timestamp, dan state CANDIDATE, ACTIVE, RETAINED, atau DELETED.

Invariant migration:

- Maksimal satu ACTIVE ObjectLocation untuk setiap ObjectRecord.
- AVAILABLE object wajib mempunyai activeObjectLocationId.
- CANDIDATE location tidak dapat melayani read.
- Verified candidate activation dan source retention terjadi dalam satu transaction.
- Temporary migration copy tidak menambah logical usedBytes.
- MigrationItem unik untuk pasangan migrationRunId dan objectRecordId.
- Tombstone delete lebih kuat daripada migration retry sehingga deleted object tidak direstorasi tanpa authorization eksplisit.

Object lifecycle:

```
RESERVED
-> UPLOADING
-> VERIFYING
-> AVAILABLE
-> DELETING
-> DELETED
```

Failure states:

```
FAILED
ABORTED
ORPHANED
RECONCILE_REQUIRED
```

Migration lifecycle:

```
PENDING
-> COPYING
-> VERIFYING
-> READY_TO_SWITCH
-> SWITCHED
-> SOURCE_DELETE_PENDING
-> COMPLETED
```

Migration control dan failure states:

```
PAUSED
RETRY_PENDING
FAILED
CANCELED
RECONCILE_REQUIRED
```

Untuk quota reservation dan counter update, gunakan serializable transaction atau equivalent locking strategy dengan retry pada transaction conflict.

Authentication dan Authorization

Dashboard

- Short-lived JWT access token.
- Refresh token dalam Secure, httpOnly, SameSite cookie.
- Refresh token rotation.
- Database menyimpan hashed refresh token.

- CSRF protection untuk cookie-authenticated mutation.
- Session revocation dan device metadata.

External storage client

- API access key dan secret generated oleh platform.
- Secret ditampilkan sekali.
- Database menyimpan hash.
- Credential terikat ke organization dan namespace.
- Prefix scopes.
- Explicit actions.
- Expiry dan rotation.
- Optional IP allowlist.
- Rate limit dan quota.

Authorization enforcement

Authorization wajib diterapkan pada:

- Backend endpoint guard sebagai source of truth.
- Service-level ownership checks.
- Repository query dengan organizationId.
- UI route dan action visibility.
- Provider calls setelah policy validation.
- Migration source dan destination ownership pada setiap control action dan worker execution.

Security

Provider credentials

- Jangan simpan secret plaintext.
- Gunakan envelope encryption dengan KMS atau secret manager.
- Pisahkan data-encryption key dan master key.
- Simpan key version untuk rotation.
- Redact access key, secret, presigned query, dan authorization headers dari log.
- Gunakan least-privilege provider credentials.

SSRF protection

Generic S3 endpoint merupakan untrusted input:

- Hanya izinkan HTTPS untuk SaaS public connector.
- Validasi hostname dan port.
- Resolve DNS dan block loopback, link-local, metadata, multicast, dan private ranges.
- Cegah DNS rebinding.
- Jangan mengikuti redirect ke lokasi terlarang.

- Terapkan network egress policy.
- Private MinIO memerlukan dedicated connector atau controlled tunnel.

File and API safety

- Maximum content length.
- Streaming; jangan buffer seluruh file.
- Content-Type tidak dianggap sebagai bukti tipe file.
- Optional antivirus/quarantine pipeline.
- Filename dan object key validation.
- CORS allowlist.
- Request rate limit.
- Idempotency protection.
- Audit security-relevant operations.

Migration safety

- Source dan destination credential hanya didekripsi di worker untuk organization yang sama.
- Worker menerima opaque identifier, bukan raw provider secret dari queue payload.
- Queue payload, error detail, tracing, dan audit tidak boleh memuat secret atau full presigned URL.
- Destination object dibuat dengan least-privilege credential dan encryption policy yang sudah lolos preflight.
- Checksum mismatch, stale source fingerprint, dan target capability mismatch harus fail closed sebelum cutover.
- Cancel hanya diizinkan sebelum cutover. Rollback setelah cutover merupakan action terpisah yang diaudit.
- Cleanup source memerlukan retained state, retention expiry, reconciliation success, dan authorized policy.

Observability

Metrics minimum:

- Upload/download bytes.
- Active streams.
- Per-provider latency dan error rate.
- Presigned versus proxied transfer count.
- Quota reservation failures.
- Placement decision count.
- Provider health.
- Orphan upload count.
- Reconciliation drift.
- Migration objects dan bytes by state.
- Migration throughput, checksum failure, retry, stale-source, cutover, rollback, dan cleanup counts.

- Retained-source bytes dan oldest retention age.
- Queue latency dan depth.
- HTTP status rate.
- OPNsense interface/pipeline utilization.

Tracing:

- requestId.
- organizationId.
- namespaceId.
- uploadId.
- migrationRunId dan migrationItemId.
- objectRecordId.
- sourceLocationId dan destinationLocationId.
- providerConnectionId.
- provider operation.

Jangan memasukkan secret, full presigned URL, atau sensitive object metadata ke tracing.

Testing Strategy

Unit tests

- Placement eligibility dan weighted selection.
- Prefix policy evaluation.
- Quota reservation.
- Object state transitions.
- Permission guards.
- Provider capability mapping.
- Migration state transitions, source fingerprint comparison, dan cutover eligibility.

Integration tests

- PostgreSQL transaction conflicts.
- Redis distributed rate limits.
- API authentication dan tenant isolation.
- Multipart completion/abort.
- Outbox processing.
- Reconciliation.
- Migration item deduplication, atomic location switch, outbox cleanup, dan worker restart resume.
- Concurrent overwrite/delete selama migration.

Provider contract tests

Jalankan test suite yang sama terhadap:

- MinIO local.
- AWS S3 sandbox.
- Cloudflare R2 sandbox.
- Backblaze B2 sandbox.

Test:

- List.
- PUT/GET/HEAD/DELETE.
- Multipart.
- Range request.
- Presigned operations.
- Error normalization.
- Special object keys.
- Large streams.
- Cross-provider stream, metadata mapping, checksum verification, dan multipart resume.

End-to-end tests

- Organization admin menambahkan provider.
- Import bucket.
- Membuat namespace dan placement target.
- Membuat folder grant dan API credential.
- Client upload, list, download, dan delete.
- Quota exceeded.
- Provider unavailable.
- Credential rotation.
- Audit log verification.
- Organization admin memigrasikan current object antar-provider tanpa logical key berubah.
- Checksum mismatch menolak cutover dan source tetap aktif.
- Partial failure, pause/resume, retry, worker restart, retained-source rollback, dan delayed cleanup.

Deployment Awal

Development:

```
Docker Compose
Next.js
Express API
Worker
PostgreSQL
Redis
MinIO
```

Production beta:

- Containerized web, API, gateway, dan worker.
- Managed atau highly available PostgreSQL.

- Managed atau highly available Redis.
- Reverse proxy di depan web dan API.
- OPNsense di network boundary bila self-hosted.
- Object traffic domain terpisah dari dashboard.
- Migration worker queue dengan independent concurrency dan backpressure.
- Automated database migrations.
- Centralized logs dan metrics.
- Database backup dan restore test.

Referensi

- Next.js self-hosting: <https://nextjs.org/docs/app/guides/self-hosting>
- Next.js Backend for Frontend: <https://nextjs.org/docs/app/guides/backend-for-frontend>
- AWS SDK JavaScript S3: <https://docs.aws.amazon.com/sdk-for-javascript/v3/developer-guide/migrate-s3.html>
- Prisma transactions: <https://www.prisma.io/docs/orm/prisma-client/queries/transactions>
- Cloudflare R2 presigned URLs: <https://developers.cloudflare.com/r2/api/s3/presigned-urls/>
- Backblaze S3-compatible API: <https://www.backblaze.com/docs/cloud-storage-s3-compatible-api>
- OPNsense traffic shaping: <https://docs.opnsense.org/manual/shaping.html>
- OPNsense SNI routing: <https://docs.opnsense.org/manual/how-tos/caddy.html>
- Product feedback and constraints: ./FEEDBACK.md